UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Operating Systems

System calls for controlling files

Mirco Marchetti, Riccardo Lancellotti

SECloud research group

University of Modena and Reggio Emilia

AY 2025/26

What We Will Cover

- File descriptors: the kernel's handle for open files
- Basic I/O system calls: `open`, `close`, `read`, `write`, `lseek`
- Advanced control with `fcntl`:
  - Querying and modifying file status flags
  - Non-blocking I/O
  - Duplicating file descriptors
  - POSIX advisory file locking
- Multiplexed I/O: monitoring multiple descriptors with `select` and `poll`

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

File Descriptors

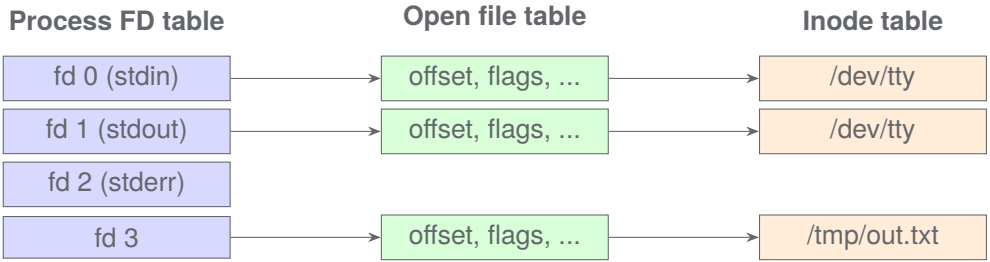
File Descriptors: Concept

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- A file descriptor (fd) is a small non-negative integer used by a process to refer to an open file (or socket, pipe, device, ...)
- It is a per-process index into the open file table maintained by the kernel
- File descriptors are the fundamental abstraction for I/O in Unix
- Every process starts with three file descriptors already open:
  - `0` — `STDIN_FILENO`: standard input
  - `1` — `STDOUT_FILENO`: standard output
  - `2` — `STDERR_FILENO`: standard error
- Declared in `<unistd.h>`

Three-Level File Table

- The kernel maintains three data structures for open files:



- Multiple fds (even in different processes) can share one open file table entry — that's how parent/child share file offsets after `fork()`

UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Basic I/O System Calls

`open()`: Opening or Creating a File

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

```c
#include <fcntl.h>
#include <sys/stat.h>

int open(const char *pathname, int flags);
int open(const char *pathname, int flags, mode_t mode);
```

- Returns a new file descriptor on success, `-1` on error (and `errno` is set)
- `pathname`: path to the file
- `flags`: bit-mask combining access mode and optional modifiers
- `mode`: permissions for newly created files (needed when `O_CREAT` is set)

`open()`: Access Mode Flags (exactly one required)

Flag	Meaning
`O_RDONLY`	Open for reading only
`O_WRONLY`	Open for writing only
`O_RDWR`	Open for both reading and writing

- Exactly one access mode must be specified; the rest are optional modifiers OR-ed together

Flag	Meaning
`O_CREAT`	Create file if it does not exist (requires `mode`)
`O_TRUNC`	Truncate file to zero length if it exists
`O_APPEND`	All writes go to end of file atomically
`O_EXCL`	With `O_CREAT`: fail if file already exists (atomic test-and-create)
`O_NONBLOCK`	Non-blocking I/O (see later)
`O_CLOEXEC`	Close fd on `exec()` (see later)

`open()`: The `mode` Argument

- The `mode` argument specifies the permission bits of a newly created file
- It is combined with the process's `umask`: actual permissions = `mode & ~umask`
- Common symbolic constants (OR them together):

Constant	Meaning
`S_IRUSR` / `S_IWUSR` / `S_IXUSR`	read / write / exec for owner
`S_IRGRP` / `S_IWGRP` / `S_IXGRP`	read / write / exec for group
`S_IROTH` / `S_IWOTH` / `S_IXOTH`	read / write / exec for others
`0644`	owner rw, group r, others r (octal)
`0600`	owner rw only (octal)

`open()`: Example — Write Log File

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

```c
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#include <stdio.h>

int main(void) {
    int fd = open("app.log",
                  O_WRONLY | O_CREAT | O_APPEND,
                  S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH);
    if (fd == -1) {
        perror("open");
        return 1;
    }
    const char *msg = "Server started\n";
    write(fd, msg, 15);
    close(fd);
    return 0;
}
```

`open()` Example: Line-by-Line

- `O_WRONLY`: we only need to write
- `O_CREAT`: create the file if it does not exist
- `O_APPEND`: every `write()` is atomic and goes to the end — safe for log files even with multiple processes
- `S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH`: permissions `0644` — owner can read/write, others can only read
- `perror("open")`: prints a human-readable error message based on `errno`
- `close(fd)`: always close fds when done — leaking descriptors can exhaust the process limit (`ulimit -n`)

`close()`: Releasing a File Descriptor

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

```c
#include <unistd.h>

int close(int fd);
```

- Returns `0` on success, `-1` on error
- Releases the file descriptor and decrements the reference count of the underlying open file table entry
- When the last reference drops to zero, kernel flushes and releases resources
- Important: always check the return value of `close()` for writable files
  - Buffered writes may be flushed only at close time — an error here means data was lost!
- File descriptors are inherited across `fork()` and preserved across `exec()` unless `O_CLOEXEC` was set

`read()`: Reading Bytes

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

```c
#include <unistd.h>

ssize_t read(int fd, void *buf, size_t count);
```

- Reads up to `count` bytes from `fd` into the buffer pointed to by `buf`
- Returns the number of bytes actually read, `0` at end-of-file, `-1` on error
- Short reads are normal: `read()` may return fewer bytes than requested
  - Network sockets, pipes, terminals — data arrives in chunks
  - Interrupted by a signal (`EINTR`)
- After a successful `read()`, the file offset advances by the number of bytes read

`read()`: Reading a File Completely

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

```c
#include <unistd.h>
#include <stdio.h>

int main(void) {
    char buf[4096];
    ssize_t n;

    int fd = open("data.bin", O_RDONLY);
    if (fd == -1) { perror("open"); return 1; }

    while ((n = read(fd, buf, sizeof(buf))) > 0) { //keep reading until EOF (0)
                                                   //or error (-1)
        /* process n bytes in buf */
        printf("Read %zd bytes\n", n);
    }
    if (n == -1) perror("read");

    close(fd);
    return 0;
}
```

`write()`: Writing Bytes

```
#include <unistd.h>

ssize_t write(int fd, const void *buf, size_t count);
```

- Writes up to `count` bytes from `buf` to `fd`
- Returns the number of bytes actually written, `-1` on error
- Short writes are possible (especially on sockets and pipes):
  - Always check the return value and loop until all bytes are written
- For regular files, a short write usually means a full disk or a system error
- If `O_APPEND` is set, each write is atomic: the seek-to-end and write happen together

`write()`: Reliable Write Helper

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

```c
#include <unistd.h>

/* Write exactly 'count' bytes; returns 0 on success, -1 on error */
int write_all(int fd, const void *buf, size_t count) {
    const char *p = buf;
    while (count > 0) {
        ssize_t n = write(fd, p, count);
        if (n == -1) {
            if (errno == EINTR) continue; /* retry on signal */
            return -1;
        }
        p += n;
        count -= n;
    }
    return 0;
}
```

- A classic idiom: loop until all bytes are consumed, handling `EINTR` explicitly

`lseek()`: Repositioning the File Offset

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

```c
#include <unistd.h>

off_t lseek(int fd, off_t offset, int whence);
```

- Moves the read/write file offset of `fd` to a new position
- Returns the resulting offset from the beginning of the file, `-1` on error
- `whence` controls how `offset` is interpreted:

whence	New offset
`SEEK_SET`	`offset` bytes from the beginning
`SEEK_CUR`	current position + `offset`
`SEEK_END`	end-of-file + `offset` (may be positive!)

- Cannot be used on pipes, sockets, or some devices — `errno` will be `ESPIPE`

`lseek()`: Common Idioms

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

```c
/* Go to the beginning */
lseek(fd, 0, SEEK_SET);

/* Get current position */
off_t pos = lseek(fd, 0, SEEK_CUR);

/* Get file size */
off_t size = lseek(fd, 0, SEEK_END);

/* Jump to byte 1024 */
lseek(fd, 1024, SEEK_SET);

/* Go back 16 bytes from current position */
lseek(fd, -16, SEEK_CUR);

/* Create a sparse file: write a byte 1 GiB in */
lseek(fd, (off_t)1 << 30, SEEK_SET);
write(fd, "\0", 1);
```

Sparse files?

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- A sparse file is a file that has unallocated blocks (holes) that read as zeros
- Created by seeking past the end and writing — the kernel allocates blocks only for the written data, not for the gap
- Saves disk space for files with large empty regions (e.g. databases, virtual machine images)
- However, not all filesystems support sparse files, and they can cause fragmentation

`lseek()`: Reading a Fixed-Size Record at Position *n*

```c
#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>

#define RECORD_SIZE 64

int read_record(int fd, int n, char *out) {
    off_t pos = (off_t)n * RECORD_SIZE;
    if (lseek(fd, pos, SEEK_SET) == -1) {
        perror("lseek"); return -1;
    }
    ssize_t r = read(fd, out, RECORD_SIZE);
    if (r != RECORD_SIZE) {
        fprintf(stderr, "Short read or EOF\n");
        return -1;
    }
    return 0;
}
```

- `lseek()` enables O(1) random access to structured binary files

UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

`fcntl`: Advanced File Control

fcntl(): Overview

```c
#include <fcntl.h>

int fcntl(int fd, int cmd, ...);
```

- Swiss-army-knife system call: performs various control operations on an open file descriptor
- The third argument depends on `cmd`
- Major command groups:
  - `F_GETFL` / `F_SETFL`: get/set file status flags
  - `F_GETFD` / `F_SETFD`: get/set file descriptor flags
  - `F_DUPFD` / `F_DUPFD_CLOEXEC`: duplicate file descriptor
  - `F_GETLK` / `F_SETLK` / `F_SETLKW`: POSIX record locking

fcntl(): Getting and Setting Status Flags


```c
/* Get current flags */
int flags = fcntl(fd, F_GETFL);
if (flags == -1) { perror("fcntl F_GETFL"); ... }

/* Add a flag (e.g. O_APPEND) without touching others */
flags |= O_APPEND;
if (fcntl(fd, F_SETFL, flags) == -1) {
    perror("fcntl F_SETFL"); ...
}

/* Remove a flag (e.g. O_APPEND) */
flags &= ~O_APPEND;
if (fcntl(fd, F_SETFL, flags) == -1) { ... }
```

- Always get before set: read the current flags, modify, then write back — avoids clobbering flags you did not intend to change
- `F_SETFL` can modify: `O_APPEND`, `O_NONBLOCK`, `O_ASYNC`
- Access mode (`O_RDONLY`/`O_WRONLY`/`O_RDWR`) cannot be changed after `open()`

Non-Blocking I/O: The Problem

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- By default, I/O system calls block the process until data is available or the write buffer has space
- This is fine for a single-threaded program handling one fd — but consider:
  - A process reading from multiple pipes and/or sockets simultaneously
  - A terminal application that must stay responsive while waiting for input
  - An event loop that must never stall
- Option 1: use multiple threads (complex, synchronization overhead)
- Option 2: set non-blocking mode with `O_NONBLOCK` and use `poll`/`select` to know when to read/write (efficient, widely used)

Setting O_NONBLOCK with fcntl()

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

```c
#include <fcntl.h>
#include <unistd.h>

/* Enable non-blocking mode on an already-open fd */
int set_nonblocking(int fd) {
    int flags = fcntl(fd, F_GETFL);
    if (flags == -1) return -1;
    return fcntl(fd, F_SETFL, flags | O_NONBLOCK);
}

/* Disable non-blocking mode */
int set_blocking(int fd) {
    int flags = fcntl(fd, F_GETFL);
    if (flags == -1) return -1;
    return fcntl(fd, F_SETFL, flags & ~O_NONBLOCK);
}
```

- Can also pass `O_NONBLOCK` directly to `open()` for new files/devices/FIFOs

Non-Blocking `read()`: Behavior and Example

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

```c
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>
#include <stdio.h>

void try_read(int fd) {
    char buf[256];
    ssize_t n = read(fd, buf, sizeof(buf));
    if (n > 0) {
        /* process n bytes */
    } else if (n == 0) {
        printf("EOF\n");
    } else if (errno == EAGAIN || errno == EWOULDBLOCK) {
        /* No data available right now -- come back later */
        printf("No data yet, try again\n");
    } else {
        perror("read");
    }
}
```

EAGAIN and EWOULDBLOCK

- `EAGAIN` / `EWOULDBLOCK`: on Linux these are the same value
- on other systems they may differ, so better check both for compatibility
- Indicates that the operation would block but the fd is in non-blocking mode, so it returns immediately instead
- When you get this error, you should stop trying to read/write and wait until the fd becomes ready again (e.g. using `select`/`poll`)
- This is a normal condition, not a failure — your program should handle it gracefully

Non-Blocking `write()`: Behavior

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

```c
ssize_t n = write(fd, buf, len);
if (n == -1) {
    if (errno == EAGAIN || errno == EWOULDBLOCK) {
        /* Kernel send/pipe buffer is full, may try again later! */
    } else {
        perror("write");
    }
} else if (n < (ssize_t)len) {
    /* Partial write: re-queue remaining bytes */
}
```

- On a non-blocking fd, `write()` returns as many bytes as were actually written, which may be less than the requested `length`
- If the buffer is completely full it returns `-1` with `EAGAIN`
- A proper event loop should wait for the fd to become “writable” again, and retry the write with the remaining data

fcntl(): Close-on-Exec Flag (FD_CLOEXEC)


```c
/* Query whether FD_CLOEXEC is set */
int fdflags = fcntl(fd, F_GETFD);
int is_cloexec = (fdflags & FD_CLOEXEC) != 0;

/* Set FD_CLOEXEC: fd will be closed automatically on exec() */
fcntl(fd, F_SETFD, fdflags | FD_CLOEXEC);

/* Equivalent: pass O_CLOEXEC to open() at creation time */
int fd2 = open("file.txt", O_RDONLY | O_CLOEXEC);
```

- `FD_CLOEXEC` causes the fd to be closed when the process calls `exec()`
- Without it, open fds are inherited by the new program
- Best practice:
  - set `O_CLOEXEC` at `open()` time
  - if a library function returned an fd that is already open, you cannot go back and add `O_CLOEXEC` to the original `open()`; in that case use `fcntl(fd, F_SETFD, ...)` to set `FD_CLOEXEC` afterward

`dup()` and `dup2()`: Duplicating File Descriptors


```c
#include <unistd.h>

int dup(int oldfd); /* returns lowest free fd */
int dup2(int oldfd, int newfd); /* makes newfd an alias */
```

- Both create a new fd that refers to the same open file table entry as `oldfd`
- The two fds share the same file offset and status flags
- `dup2()` is used for I/O redirection: `dup2(filefd, STDOUT_FILENO)` makes standard output go to a file
- `fcntl(fd, F_DUPFD, minfd)`: like `dup()` but guarantees the new fd is $\geq$ `minfd`

dup2(): Redirect stdout to a File

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

```c
#include <fcntl.h>
#include <unistd.h>

int main(void) {
    int logfd = open("output.log",
                     O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (logfd == -1) { perror("open"); return 1; }

    /* Redirect stdout (fd 1) to the log file */
    if (dup2(logfd, STDOUT_FILENO) == -1) {
        perror("dup2"); return 1;
    }
    close(logfd); /* original fd no longer needed */

    printf("This goes to output.log\n");
    return 0;
}
```

- After `dup2()`, `logfd` and `STDOUT_FILENO` both point to the same open file entry
- Closing `logfd` does not affect `STDOUT_FILENO`

POSIX Advisory Locking: Concepts

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Advisory locking: the kernel enforces locks only between cooperating processes that both call `fcntl()` — it does not prevent uncooperating processes from accessing the file
- Two lock types:
  - `F_RDLCK` (shared / read lock): multiple readers can coexist; incompatible with write locks
  - `F_WRLCK` (exclusive / write lock): only one writer; incompatible with all other locks
  - `F_UNLCK`: release the lock
- Locks are associated with (PID, file) pairs — not with individual file descriptors
  - Opening the same file twice from the same process creates two fds but they share the same lock context; locking via one removes locks via the other

POSIX File Locking: `struct flock`

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

```
struct flock {
    short l_type; /* F_RDLCK, F_WRLCK, F_UNLCK */
    short l_whence; /* SEEK_SET, SEEK_CUR, SEEK_END */
    off_t l_start; /* starting offset */
    off_t l_len; /* number of bytes (0 = to EOF) */
    pid_t l_pid; /* set by F_GETLK: owner PID */
};
```

- `fcntl(fd, F_SETLK, &fl)`: try to set/release lock; returns `-1/EACCES` if blocked
- `fcntl(fd, F_SETLKW, &fl)`: wait (block) until lock is available (*W* = wait)
- `fcntl(fd, F_GETLK, &fl)`: test if a lock would be blocked; `l_type` becomes `F_UNLCK` if not conflicting, otherwise set to the blocking lock
- Locks span byte ranges — you can lock individual records in a shared database file

File Locking Example: Exclusive Write Lock


```c
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int lock_file(int fd) {
    struct flock fl = {
        .l_type = F_WRLCK,
        .l_whence = SEEK_SET,
        .l_start = 0,
        .l_len = 0, /* 0 = lock entire file */
    };
    return fcntl(fd, F_SETLKW, &fl);
}

int unlock_file(int fd) {
    struct flock fl = {
        .l_type = F_UNLCK,
        .l_whence = SEEK_SET,
        .l_start = 0,
        .l_len = 0,
    };
```

File Locking Example: Usage

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

```c
int fd = open("shared.db", O_RDWR);
if (fd == -1) { perror("open"); return 1; }

/* Wait until we can get an exclusive lock */
if (lock_file(fd) == -1) { perror("lock"); return 1; }

/* --- critical section: read, modify, write --- */
char buf[128];
read(fd, buf, sizeof(buf));
/* ... modify buf ... */
lseek(fd, 0, SEEK_SET);
write(fd, buf, sizeof(buf));
/* ------------------------------------------- */

if (unlock_file(fd) == -1) { perror("unlock"); }
close(fd);
```

- `F_SETLKW` blocks until no other process holds a conflicting lock
- Locks are automatically released when the fd is closed or the process exits

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Multiplexed I/O: `select` and `poll`

Why Multiplexed I/O?

- A process often needs to monitor multiple file descriptors simultaneously
- Options:
  - Blocking: `read()` one fd at a time — misses events on others
  - Non-blocking polling: loop through all fds calling `read()` — wastes CPU
  - Multiplexed I/O: tell the kernel *"wake me when any of these fds is ready"*
- Two classic POSIX interfaces:
  - `select()`: oldest; limited to fd < 1024
  - `poll()`: newer; no hard fd limit
- Modern Linux also offers `epoll` (out of scope here)

`select()`: Prototype

```c
#include <sys/select.h>

int select(int nfds,
           fd_set *readfds,
           fd_set *writefds,
           fd_set *exceptfds,
           struct timeval *timeout);

/* fd_set manipulation macros */
FD_ZERO(fd_set *set); /* clear all bits */
FD_SET(int fd, fd_set *set); /* add fd to set */
FD_CLR(int fd, fd_set *set); /* remove fd from set */
FD_ISSET(int fd, fd_set *set); /* test fd in set */
```

- `nfds`: highest fd number + 1
- Returns the number of ready fds, `0` on timeout, `-1` on error
- On return, the sets are modified: only ready fds remain set — rebuild every call!

`select()`: Parameters in Detail

Parameter	Meaning
`readfds`	Fds to watch for data available to read
`writefds`	Fds to watch for space available to write
`exceptfds`	Fds to watch for exceptional conditions (e.g., out-of-band TCP data)
`timeout`	Maximum time to wait; `NULL` = block forever; `{0,0}` = return immediately

- `struct timeval` has two fields: `tv_sec` (seconds) and `tv_usec` (microseconds)
- POSIX allows `select()` to modify `timeout`: always re-initialize it each iteration
- Hard limit: `FD_SETSIZE` is typically 1024 — fds $\geq$ 1024 overflow the bit set silently!

`select()`: Complete Example

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

```c
#include <sys/select.h>
#include <unistd.h>
#include <stdio.h>

int main(void) {
    fd_set rfds;
    struct timeval tv;

    FD_ZERO(&rfds);
    FD_SET(STDIN_FILENO, &rfds); /* watch stdin */

    tv.tv_sec = 5; tv.tv_usec = 0; /* 5-second timeout */

    int ret = select(STDIN_FILENO + 1, &rfds, NULL, NULL, &tv);
    if (ret == -1) perror("select");
    else if (ret == 0) printf("Timeout: no input in 5s\n");
    else if (FD_ISSET(STDIN_FILENO, &rfds))
        printf("stdin is readable\n");

    return 0;
}
```

`select()`: Monitoring Multiple Fds

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

```c
int fd1 = open("fifo1", O_RDONLY | O_NONBLOCK);
int fd2 = open("fifo2", O_RDONLY | O_NONBLOCK);
int maxfd = (fd1 > fd2 ? fd1 : fd2) + 1;

while (1) {
    fd_set rfds;
    FD_ZERO(&rfds);
    FD_SET(fd1, &rfds);
    FD_SET(fd2, &rfds);

    if (select(maxfd, &rfds, NULL, NULL, NULL) == -1) {
        perror("select"); break;
    }
    if (FD_ISSET(fd1, &rfds)) { /* read from fd1 */ }
    if (FD_ISSET(fd2, &rfds)) { /* read from fd2 */ }
}
```

- Rebuild `rfds` from scratch on every iteration (the kernel clears non-ready bits)

poll(): Prototype

```c
#include <poll.h>

int poll(struct pollfd *fds, nfds_t nfds, int timeout);

struct pollfd {
    int fd; /* file descriptor to watch */
    short events; /* requested events (input) */
    short revents; /* returned events (output) */
};
```

- `nfds`: number of entries in the `fds` array
- `timeout`: milliseconds to wait; `-1` = block forever; `0` = return immediately
- Returns the number of fds with non-zero `revents`, `0` on timeout, `-1` on error
- No hard fd-number limit; works with any fd number

poll(): Event Flags

Flag	In events?	Meaning
POLLIN	yes	Data available to read
POLLOUT	yes	Space available to write without blocking
POLLPRI	yes	Urgent / out-of-band data
POLLERR	no	Error condition (always reported)
POLLHUP	no	Hang-up (peer closed connection); always reported
POLLNVAL	no	fd is not open; always reported

- POLLERR, POLLHUP, and POLLNVAL are always reported in revents regardless of events
- Check revents after poll() returns; do not assume revents == events

poll(): Complete Example

```c
#include <poll.h>
#include <unistd.h>
#include <stdio.h>

int main(void) {
    struct pollfd fds[2];
    fds[0].fd = STDIN_FILENO; fds[0].events = POLLIN;
    fds[1].fd = STDOUT_FILENO; fds[1].events = POLLOUT;

    int ret = poll(fds, 2, 3000); /* 3-second timeout */
    if (ret == -1) { perror("poll"); return 1; }
    if (ret == 0) { printf("Timeout\n"); return 0; }

    if (fds[0].revents & POLLIN)
        printf("stdin readable\n");
    if (fds[1].revents & POLLOUT)
        printf("stdout writable\n");
    if (fds[0].revents & (POLLERR | POLLHUP | POLLNVAL))
        printf("stdin error\n");
    return 0;
}
```

`poll()`: Event Loop Skeleton

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

```c
#define NFDS 4
struct pollfd fds[NFDS];
/* ... populate fds[i].fd and fds[i].events ... */

while (1) {
    int n = poll(fds, NFDS, -1); /* block until event */
    if (n == -1) {
        if (errno == EINTR) continue; /* signal, retry */
        perror("poll"); break;
    }
    for (int i = 0; i < NFDS; i++) {
        if (fds[i].revents == 0) continue;
        if (fds[i].revents & POLLIN) handle_read(fds[i].fd);
        if (fds[i].revents & POLLOUT) handle_write(fds[i].fd);
        if (fds[i].revents & POLLHUP) handle_close(fds[i].fd);
    }
}
```

- Unlike `select()`, the `fds` array is not destroyed by `poll()`: only `revents` is updated

`select()` vs `poll()`: Comparison

Feature	`select()`	`poll()`
Data structure	Bit mask (`fd_set`)	Array of `struct pollfd`
Max fd number	`FD_SETSIZE` (1024)	Unlimited
Input set reuse	No: rebuilt every call	**Yes**: only `revents` changes
Timeout precision	Microseconds	Milliseconds
Portability	Very high (even old POSIX)	POSIX.1-2001
Distinguishes error types	No	**Yes** (`POLLERR`/`POLLHUP`/`POLLNVAL`)
Scalability	Poor (O(FD_SETSIZE))	Poor (O(n)), but no 1024 cap

- For new code, prefer `poll()` over `select()`
- For very high fd counts ($>$ a few thousand) use `epoll` (Linux) or `kqueue` (BSD/macOS)

Summary

- File descriptors are the universal I/O abstraction in Unix — integers indexing kernel structures
- Basic I/O: `open`/`close`/`read`/`write`/`lseek` cover the vast majority of use cases
  - Always handle short reads/writes; always check return values
- `fcntl()` provides fine-grained control:
  - `F_GETFL`/`F_SETFL`: modify flags (e.g., `O_NONBLOCK`, `O_APPEND`)
  - `F_GETFD`/`F_SETFD`: manage `FD_CLOEXEC`
  - `F_SETLK`/`F_SETLKW`: POSIX byte-range advisory locking
- Lockfiles (`O_CREAT|O_EXCL`) provide atomic singleton protection
- Multiplexed I/O: `select()` and `poll()` let one thread wait on many fds efficiently
  - `poll()` is preferred: no fd-number limit and no input-set destruction